

BTS 2 – 2024/2025

Projet 2 : Création d'un Serveur Web en Java – Deuxième partie

Rappel

Ce projet est à réaliser seul ou en binôme, **une documentation détaillée** est demandée, celle ci devra :

- Présenter votre projet
- Répondre aux **questions** ouvertes dans ce document
- Expliquer comment fonctionne votre projet de façon détaillée, un schéma d'enchaînement des méthodes appelées peut être utile.
- Pour chaque version réalisée (0.1, 0.2, etc.) vous mettrez en annexe de votre documentation l'intégralité du code correspondant à cette version.
- Donner toutes les références externes utilisées (liens vers les documentations, liens vers le protocole, liste des codes http implémentés etc.).

Consignes

Vos noms de propriétés doivent être en anglais et explicites. Les noms de vos méthodes doivent être en anglais et basées sur un verbe et décrire de façon explicite ce que fait la méthode.

Vous utiliserez la notation standard suivante :

Classes : camelCase avec première lettre en majuscule.

Propriétés : camelCase avec première lettre en minuscule

Méthodes : camelCase avec première lettre en minuscule

Faites attention à la comparaison de chaînes de caractères en Java.

*L'ensemble de votre code devra être en mesure de **gérer les exceptions** pour cela vous utiliserez la commande try !*

Etape 3 : Rendre le projet viable

Introduction

A l'issue des deux premières étapes vous avez réalisé une ébauche de ce que sera votre serveur web, vous allez maintenant le faire évoluer pour en faire un outil viable et utilisable au delà d'une simple preuve de concept !

Pour cela vous allez devoir étendre et refactoriser le code déjà réalisé afin de le rendre totalement fonctionnel.

Actuellement votre serveur web se compose de deux classes `WebServerApplication` et `WebServer`, la première sert à initialiser l'environnement de votre application et à lancer votre serveur web, le lancement ouvre le port 8080 et fait que votre application attend de recevoir une requête http sur le port 8080, une fois une requête reçue celle ci est traitée par la méthode `readRequest()` et une réponse (les entêtes http et le contenu d'un fichier html) est renvoyé au client par la méthode `sendResponse()`.

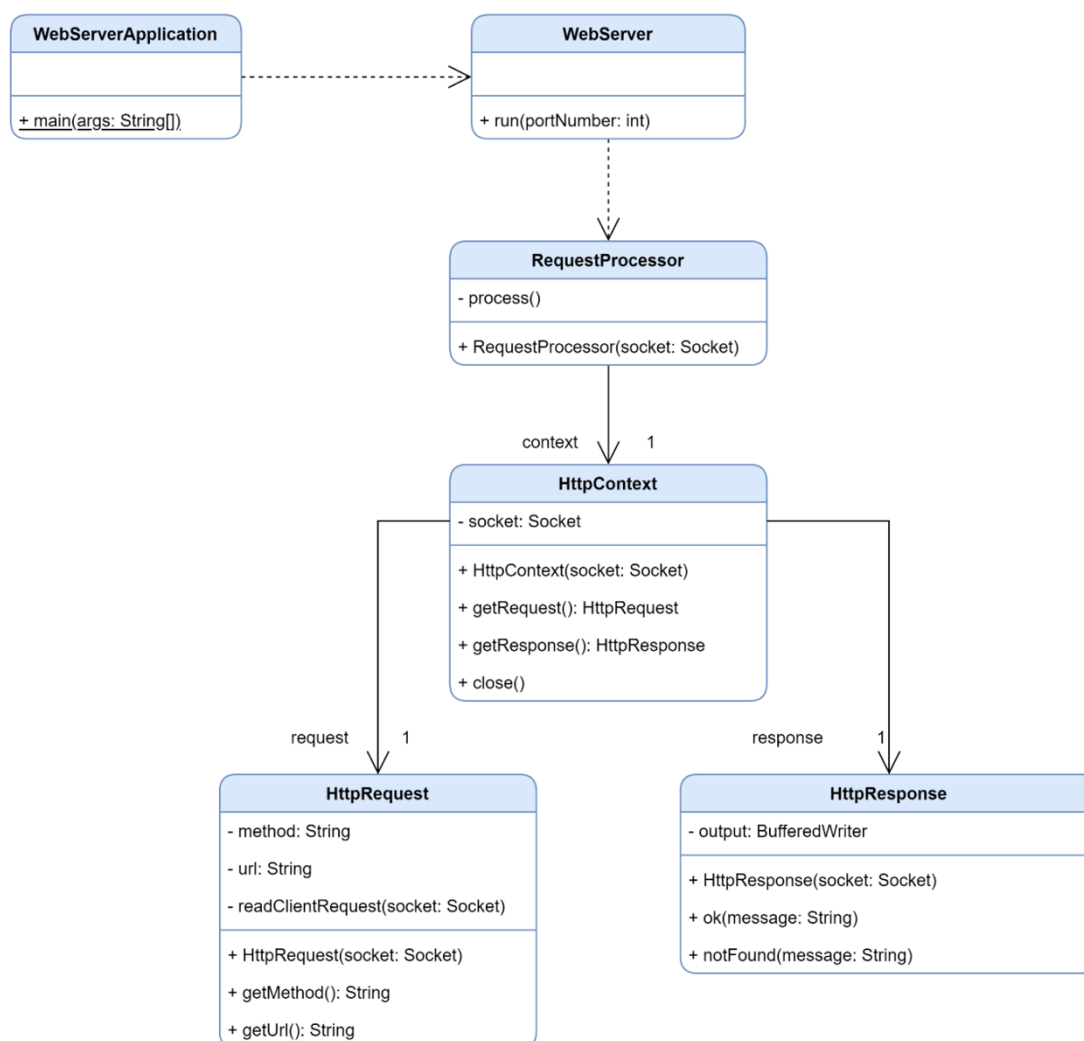
Le protocole http ne s'arrête pas à la requête GET, il doit aussi être capable de traiter les requêtes de type POST, il est à noter qu'il existe d'autres types de requêtes définies au point 9.1.2 de la RFC 2616 mais nous ne les traiterons pas dans ce projet.

De la même façon un serveur web ne se contente pas de renvoyer un code 200 pour dire que tout est OK, il doit aussi renvoyer les autres codes de retour tels que définis au point 10 de la RFC 2616, dans le cadre de ce projet vous vous contenterez de traiter les cas : 200, 400, 404 et 503.

Réorganisation de la structure du modèle de l'application (version 0.3)

Vous allez maintenant pouvoir réorganiser votre code afin d'obtenir le schéma suivant, ne paniquez pas, ça peut sembler plus complexe mais il n'y a que 4 classes et la majorité du code demandé a déjà été écrit dans la version 0.2 !

Dans cette version on se contentera de renvoyer les headers http, c'est dans la version 0.4 que l'on renverra le contenu demandé !



Description des classes et des méthodes de la version 0.3

HttpRequest

La classe HttpRequest sera chargée de lire les informations transmises par le client et d'en extraire les éléments utiles.

Implémentez la classe HttpRequest

La méthode readClientRequest lira les informations en provenance du client et extraira la méthode HTTP (GET, POST, PUT, ...) utilisée et l'URL demandée.

HttpResponse

La classe HttpResponse sera utilisée pour transmettre des informations au client.

Implémentez la classe HttpResponse

La méthode ok indiquera au client que sa requête a bien été traitée

HTTP/1.0 200 Message

La méthode notFound indiquera au client que la ressource demandée n'a pas été trouvée :

HTTP/1.0 404 Message

HttpContext

La classe HttpContext gèrera la connexion avec le client et fournira un accès aux éléments de la requêtes ainsi qu'aux fonctionnalités de réponse.

Implémentez la classe HttpContext

La méthode close se chargera de fermer la connexion.

RequestProcessor

La classe RequestProcessor s'occupera de traiter une requête d'un client.

Implémentez la classe RequestProcessor

La méthode process renverra un code 200 au client si l'url demandée est / et un code 404 dans les autres cas.

WebServer

Les méthodes readRequest et sendResponse ne sont plus d'actualité et doivent être supprimées.

La méthode run est invoquée par la méthode main de votre application c'est ici que vous créez une instance de la classe RequestProcessor qui traitera la requête du client à chaque fois qu'une connexion est acceptée.

Modifiez la classe WebServer pour qu'elle corresponde à ce fonctionnement.

A la fin de cette étape, avec la version 0.3, vous devez avoir le squelette de ce que sera votre application avec un fonctionnement proche de ce que l'on attend d'un serveur web ! La version suivante sera un serveur web complet capable de traiter une requête http et d'y apporter la réponse demandée, pas seulement au niveau des headers mais aussi au niveau du contenu !

Etape 4 : Terminer le projet

A ce stade il nous reste à ajouter quelques petites choses pour que notre projet soit terminé :

- Gérer le contenu, ce sera la version 0.4
 - o trouver le fichier demandé s'il existe
 - o renvoyer les bons headers (code de retour, type de fichier, etc.)
 - o renvoyer le contenu du fichier demandé, avec le bon type MIME
- Etre capable de gérer plusieurs requêtes en mettant en place le multithreading, ce sera la version 0.5

Une fois tout cela terminé et testé vous pourrez considérer votre projet terminé et le passer en version 1.0 qui sera la version finale du projet.

Gestion du contenu (version 0.4)

Tout d'abord vous allez ajouter une méthode `sendContent(contentType : String, content : String)` à la classe `HttpResponse`

Cette méthode prendra en paramètre le type de contenu et le renverra dans les headers http, par exemple `text/plain`, elle renverra aussi le contenu sous la forme d'un `String`.

Pour tester modifiez le code de la méthode `process` de la classe `RequestProcessor` pour envoyer le texte « hello world » avec le type `text/plain` après avoir envoyé le code 200. Ensuite modifiez le code de la méthode `process` mais cette fois ci envoyez le avec le type `text/html`

Est-ce que vous voyez une différence selon le type renvoyé, ajoutez du html dans votre texte renvoyé (par exemple avec une balise `<b></b>` autour de Hello World et regardez s'il y a une différence !

Une fois le fonctionnement de l'envoi du type mime testé et validé vous allez pouvoir ajouter le traitement d'un fichier

Ajoutez une méthode `sendFile(contentType : String, fileName : String)` à la classe `HttpResponse`

Cette méthode prend en paramètre le type de contenu transmis ainsi que le nom du fichier dont le contenu sera transmis par la méthode `sendContent` ajoutée ci dessus.

Je vous conseille de traiter tous les fichiers comme des fichiers binaires pour pouvoir traiter avec la même méthode tous les types de fichier qu'ils soient de type text (comme HTML ou CSS, etc.) ou que ce soit des images (comme jpg, png, etc.)

Pour cela il faudra modifier la propriété `output` de `HttpResponse` et lui donner le type `OutputStream` plutôt que `BufferedWriter`. Regardez la documentation java de `getBytes` de la classe `String` pour vous aider ;)

Créez un dossier que vous nommerez « public » à la racine de votre projet et placez y un fichier `index.html` (vous pouvez reprendre la page html de la version 0.2). Faites en sorte (comme dans la version 0.2) de modifier la méthode `process` de la classe `RequestProcessor` afin de bien transmettre le bon type MIME (`text/html`) et le contenu du fichier `index.html` lorsque l'url demandée est `/`.

A l'issue de cette modification la méthode process de RequestProcessor doit :

- Transmettre un code 200 et le fichier index.html si l'url demandée est /
- Transmettre un code 200 et le fichier demandé s'il existe dans le dossier public
- Transmettre un code 404 et un message si le fichier n'existe pas dans public
- Fournir le bon type MIME dans l'entête Content-type en fonction du fichier renvoyé en se basant sur son extension

Vous trouverez la liste des codes MIME correspondant aux extensions de fichier les plus courants ici :

https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types/Common_types

Implémentez les plus courants : html, css, png)

Testez le bon fonctionnement avec :

- Un fichier html simple
- Un fichier html qui appelle un fichier css
- Un fichier image au format png
- Une page qui contient une image et appelle un fichier css

Bravo, votre application est maintenant un vrai serveur web. Il ne reste plus qu'à la rendre multi-tâches pour pouvoir traiter plusieurs requêtes « simultanément ».

Ajouter le multi-tâches (version 0.5)

Si vous affichez dans la console ce qui se passe vous remarquerez que lorsque vous appelez la page qui contient du html et une image les 3 requêtes (celle pour la page html, celle pour la feuille de style et celle pour l'image) sont exécutées l'une après l'autre. Pour une page simple ce n'est pas très grave mais imaginez les temps d'attente si plusieurs personnes consultent la même page en même temps ou que la page contient beaucoup d'éléments .

Pour rendre une application multitâches Java met à votre disposition l'implémentation de l'interface « Runnable », pour la déployer il vous faudra modifier le code de WebServer pour qu'un thread soit créé à chaque connexion plutôt qu'un seul thread ne s'exécute.

La documentation de Java ainsi que de nombreux exemples en ligne vous aideront à trouver une solution !

Testez bien que chaque nouvelle connexion ouvre un nouveau thread, assurez vous bien aussi que chaque nouvelle connexion soit bien terminée pour éviter un dépassement de la mémoire de votre ordinateur !

Une fois les tests réalisés et validés vous avez terminé ce projet, vous pouvez lui donner un numéro de version 1.0 pour montrer que c'est une version complète !